

Documento A — Base Conceitual de Teste

Projeto: DietHub

Tecnologia: Flutter + Node.js + MySQL

Arquitetura: MVVM + API REST

Norma aplicada: ISO/IEC/IEEE 29119-1

1. Sistema sob Teste

O sistema sob teste consiste no aplicativo móvel DietHub, desenvolvido em Flutter, responsável pelo gerenciamento nutricional do usuário. O aplicativo permite:

- Cadastro e autenticação de usuários
- Geração de planos alimentares personalizados
- Registro e acompanhamento de refeições
- Navegação entre módulos do sistema
- Comunicação com APIs externas
- Controle de estado da aplicação
- Processamento de dados nutricionais

A aplicação utiliza arquitetura MVVM integrada a serviços REST e gerenciamento de estado.

2. Itens de Teste

Camada de Apresentação

- LoginScreen
- RegisterScreen
- HomeScreen
- CreatePlanScreen
- ScanPlanScreen
- LogMealScreen

- MealHistoryScreen
- ProfileScreen

Camada de Estado

- AppState
- Providers
- ViewModels

Camada de Serviços

- ApiService
- AIService
- AuthService
- NutritionService

Fluxos de Integração

- Fluxo Cadastro → Login
- Fluxo Login → Home
- Fluxo Home → Plano Alimentar
- Fluxo Registro de Refeição
- Fluxo Scan → Leitura Nutricional

3. Escopo

Os testes contemplam:

- Validação de autenticação
- Cadastro de usuários
- Login e logout
- Navegação entre telas
- Geração de planos alimentares
- Registro de refeições
- Comunicação com APIs
- Atualização de estado global
- Exibição correta de dados nutricionais

- Tratamento de erros
- Validação de campos obrigatórios
- Fluxos principais do sistema
- Testes automatizados
- Testes funcionais
- Testes de integração

4. Fora de Escopo

Não fazem parte desta etapa:

- Testes de carga
- Testes de stress
- Segurança avançada
- Pentest
- Performance de produção
- Firebase real
- Banco MySQL em produção
- Publicação em lojas mobile
- Compatibilidade com iOS
- Testes offline

5. Requisitos Funcionais

- **RF01** — O sistema deve permitir cadastro de usuários.
- **RF02** — O sistema deve validar campos obrigatórios.
- **RF03** — O sistema deve impedir e-mails inválidos.
- **RF04** — O sistema deve impedir login inválido.
- **RF05** — O sistema deve autenticar usuários válidos.
- **RF06** — O sistema deve navegar para Home após login.
- **RF07** — O sistema deve gerar plano alimentar personalizado.
- **RF08** — O sistema deve registrar refeições.
- **RF09** — O sistema deve exibir histórico nutricional.
- **RF10** — O sistema deve atualizar o estado global da aplicação.
- **RF11** — O sistema deve consumir APIs REST.
- **RF12** — O sistema deve permitir leitura nutricional via Scan.

- **RF13** — O sistema deve exibir mensagens de erro.
- **RF14** — O sistema deve permitir logout.

6. Condições de Teste

- **CT01** — Validar cadastro válido.
- **CT02** — Validar cadastro com campos vazios.
- **CT03** — Validar e-mail inválido.
- **CT04** — Validar login válido.
- **CT05** — Validar login inválido.
- **CT06** — Validar navegação para Home.
- **CT07** — Validar geração de plano alimentar.
- **CT08** — Validar exibição do plano alimentar.
- **CT09** — Validar registro de refeições.
- **CT10** — Validar atualização do AppState.
- **CT11** — Validar comunicação com API.
- **CT12** — Validar funcionalidade de Scan.
- **CT13** — Validar fluxo completo de autenticação.
- **CT14** — Validar mensagens de erro.

7. Tipos de Teste

- Teste de Unidade
- Teste Funcional
- Teste de Integração
- Teste Baseado em Cenário
- Teste de Regressão
- Teste de Validação

8. Riscos

- **R01** — Login inválido permitir acesso.
- **R02** — APIs retornarem dados inconsistentes.
- **R03** — Estado da aplicação ficar inconsistente.
- **R04** — Navegação entre telas falhar.
- **R05** — Plano alimentar não ser gerado.

- **R06** — Registro de refeições falhar.
- **R07** — Scan nutricional apresentar leitura incorreta.
- **R08** — Mensagens de erro não serem exibidas.
- **R09** — Perda de sessão do usuário.